

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Information and Communication Technology



Vietnamese Legal RAG

Project I

Submitted by:

Name: Dang Xuan Bach

ID: 202416664

Email: bach.dx2416664@sis.hust.edu.vn

Supervisor: Tran Thi Lan Anh

June 30, 2026

Contents

Chapter 1. Introduction	5
1.1 Motivation	5
1.2 Problem Statement	5
1.3 Objectives	5
1.4 Contributions	6
Chapter 2. Dataset	6
2.1 Dataset Overview	6
2.2 Dataset Statistics	6
2.3 Legal Sector Filtering	7
2.4 Data Preprocessing	8
Chapter 3. Overall System Pipeline	11
3.1 Configured Data Flow	12
3.2 Build Workflow	12
3.3 Retrieval and Evaluation Workflow	12
3.4 System Modes	13
Chapter 4. Retrieval Methods and Mathematical Formulation	13
4.1 Retrieval-Augmented Generation Principle	13
4.2 Dense Vector Retrieval	13
4.3 BM25 Lexical Retrieval	14
4.4 Weighted Reciprocal Rank Fusion	14
4.5 Legal Graph Construction	15
4.6 Graph-Only Expansion	15
4.7 Hybrid Plus Graph Retrieval	16
4.8 Evaluation Objective	16
Chapter 5. Data Preprocessing Pipeline	17
5.1 Dataset Pairing and Input Validation	17
5.2 Legal-Sector Filtering	17
5.3 Two-Level Text Normalisation	17
5.4 Document Header Processing	18
5.5 Line-by-Line Legal Structure Parser	18
5.6 Article Chunk Creation	18
5.7 Administrative Tail Removal	19
5.8 Appendix Detection and State Handling	19
5.9 Table-Aware Appendix Splitting	19
5.10 Legal Reference and Relationship Extraction	20
5.11 Output Chunk Schema	20
5.12 Relation to Index Builders and Graph Mapping	21
Chapter 6. Dense Passage Embedding and Vector Store Construction	21
6.1 Embedding Model	21
6.2 Device and Precision	21
6.3 Batch Encoding	21
6.4 FAISS Index	22

6.5	BM25 Index Construction	22
Chapter 7.	Legal Knowledge Graph Construction	22
7.1	Node Types	22
7.2	Hierarchy Edges	23
7.3	Document Reference Edges	23
7.4	Chunk-to-Node Mapping	23
7.5	Graph Traversal and Propagation	24
Chapter 8.	Retrieval Runtime Architecture	24
8.1	BM25 Retrieval	24
8.2	Vector Retrieval	24
8.3	Component Caching	24
8.4	Application Entry Point	24
Chapter 9.	Answer Generation	25
9.1	Generation Model	25
9.2	Grounded Generation	25
9.3	Separation of Concerns	25
Chapter 10.	Evaluation Framework	25
10.1	Evaluation Queries	25
10.2	Evaluation Cutoffs	26
10.3	Hit@K	26
10.4	Recall@K	26
10.5	MRR@K	27
10.6	Miss Count	27
10.7	Graph Diagnostic Counts	28
10.8	Query-Level Diagnostics	28
Chapter 11.	Hyperparameter Tuning	28
11.1	Hybrid Search Grid	28
11.2	Graph Grid	28
11.3	Hybrid-Graph Grid	29
11.4	Selection Objective	29
11.5	Number of Tuning Trials	30
11.6	Tuning Procedure	30
Chapter 12.	System Implementation	30
12.1	Class-Based Architecture	30
12.2	Module Responsibilities	31
12.3	Implementation Notes	31
12.4	Output Files	31
12.5	Reproducibility Controls	32
Chapter 13.	Results and Analysis	32
13.1	Quantitative Results	32
13.2	Best Hyperparameters	33
13.3	Method Interpretation	33

13.4 Shallow Versus Deep Cutoffs	34
13.5 Miss Analysis	34
13.6 Graph Error Analysis	34
13.7 Hyperparameter Interpretation	34
13.8 Failure Mode Examples	34
Chapter 14. Limitations	35
14.1 Hyperparameter Tuning and Evaluation on the Same Queries	35
14.2 Graph Granularity	35
14.3 Limited Relation Semantics	35
14.4 No Learned Reranker	35
14.5 Synthetic or Automatically Constructed Queries	35
14.6 Computational Cost	35
Chapter 15. Further Improvements	36
15.1 Separate Tuning and Test Sets	36
15.2 Cross-Encoder Reranking	36
15.3 Domain-Adaptive Fine-Tuning	36
15.4 Relation-Aware Graph Scoring	36
15.5 Query-Adaptive Graph Expansion	36
15.6 Graph Pruning	36
15.7 Approximate Vector Indexing	37
15.8 Real-User Evaluation	37
Chapter 16. Author Contributions	37
Chapter 17. Conclusion	37

Abstract

Vietnamese legal documents contain long hierarchical structures, domain-specific terminology, exact legal identifiers, appendices, tables, and cross-document references. These properties make legal information retrieval difficult for systems based only on keyword matching or semantic similarity. This project develops a class-based Vietnamese Legal Retrieval-Augmented Generation system that integrates lexical retrieval, dense vector retrieval, rank fusion, and legal graph expansion in a unified and reproducible pipeline.

The current system uses Qwen3-Embedding-0.6B to encode legal chunks and queries, FAISS for dense nearest-neighbour search, BM25 for lexical retrieval, and weighted Reciprocal Rank Fusion for hybrid ranking. A legal hierarchy graph maps legal documents into document, part, chapter, section, and article nodes, and adds document-level reference edges from extracted legal citations. Graph propagation can therefore expand and rerank candidates using structural and citation context around article-level chunks.

The experimental pipeline evaluates five methods: BM25, vector retrieval, graph retrieval, hybrid retrieval, and hybrid retrieval combined with graph expansion. Hyperparameters are tuned automatically over 4,000 evaluation queries. Performance is measured at $K \in \{5, 10, 50, 100, 200\}$ using Hit@K, Recall@K, and Mean Reciprocal Rank at K. The framework also records the number of missed queries, graph mapping failures, and empty graph expansions. The final implementation produces both a method-level comparison table and query-level diagnostic results, enabling systematic analysis of retrieval quality and graph behaviour.

Chapter 1. Introduction

1.1 Motivation

Vietnamese legislation is expressed through laws, decrees, circulars, decisions, official guidance documents, and local-government regulations. A single legal question may require locating an exact article number, interpreting a semantically equivalent provision, or following a relationship between several documents. Conventional keyword search is effective when the user knows the exact legal wording, but it often fails for paraphrased or scenario-based queries. Dense retrieval improves semantic matching, although it may overlook exact identifiers, rare legal expressions, and structured references.

Retrieval-Augmented Generation (RAG) addresses this limitation by retrieving external evidence before generating an answer [2]. In the legal domain, retrieval quality is particularly important because an answer generator should be grounded in authoritative provisions rather than relying only on its parametric memory. The retrieval component must therefore balance semantic similarity, lexical precision, structural context, and legal relationships.

1.2 Problem Statement

The project addresses four main problems:

1. Legal chunks contain both semantic content and exact identifiers, making either dense-only or lexical-only retrieval incomplete.
2. Retrieval operates at chunk level, while legal structure and references often exist at document or hierarchy level.
3. Graph construction must map article-level retrieval chunks into a legal hierarchy and resolve document-level references extracted from preprocessing.
4. Retrieval methods and graph parameters must be compared under a common evaluation framework using the same queries, cutoffs, and relevance labels.

1.3 Objectives

The objectives of the revised system are:

- Build a reusable class-based software architecture with a single application entry point.
- Encode Vietnamese legal chunks using Qwen3-Embedding-0.6B.
- Construct FAISS and BM25 indexes over the same ordered corpus.
- Compare BM25, vector, graph, hybrid, and hybrid-plus-graph retrieval.
- Construct a document-part-chapter-section-article legal hierarchy graph with document-reference edges.
- Tune retrieval and graph hyperparameters automatically.
- Evaluate 4,000 queries at $K = \{5, 10, 50, 100, 200\}$.

- Export both aggregate and query-level results for error analysis.

1.4 Contributions

The project makes the following contributions:

- A clean object-oriented implementation of a Vietnamese legal retrieval pipeline.
- A unified retrieval interface for five different methods.
- A graph builder that creates stable hierarchy nodes and document-reference edges from preprocessed legal chunks.
- A cached hyperparameter-tuning procedure that avoids repeatedly encoding the same query.
- A multi-cutoff evaluation framework reporting Hit@K, Recall@K, MRR@K, and misses.
- Reproducible CSV and JSON outputs suitable for subsequent statistical analysis and reporting.

Chapter 2. Dataset

2.1 Dataset Overview

The original data source is the public Vietnamese legal-document dataset [1]:

`minhdoan17/vietnamese-legal-documents`.

This dataset is a large Vietnamese legal corpus containing documents from many legal domains and document types, such as laws, decrees, circulars, decisions, and administrative regulations. Each document provides legal text together with metadata fields such as document identifier, title, issuing authority, legal sector, document type, issuance date, and source URL when available. These properties make the dataset suitable for information retrieval, legal question answering, and Retrieval-Augmented Generation (RAG) experiments.

In this project, the full dataset is not indexed directly. Due to limited time and computational resources, the experiment focuses on a smaller subset in the education sector. This choice keeps the project feasible while still preserving the main characteristics of Vietnamese legal retrieval: long formal documents, specialised legal terminology, exact article references, appendices, tables, and metadata-rich records. Therefore, all experimental results in this report should be interpreted as education-domain LegalRAG results rather than a full benchmark over the entire Vietnamese legal corpus.

2.2 Dataset Statistics

Let the original legal corpus be denoted as:

$$D = \{(x_i, s_i, m_i)\}_{i=1}^N,$$

where x_i is the raw legal document text, s_i is the legal sector label, m_i is the metadata associated with the document, and N is the number of documents in the original dataset. The corpus covers multiple sectors, so indexing all documents would require substantially more embedding time, disk space, and retrieval memory.

After filtering and preprocessing, the working corpus used by the retrieval system is stored as:

`data/legal_chunks.parquet`.

Each row in this file is a retrieval chunk rather than a full document. The main fields used by the system are shown in Table 1.

Table 1: Main fields in the preprocessed education-domain chunk corpus.

Field	Description
<code>id</code>	Unique chunk identifier used by retrieval and evaluation
<code>document_id</code>	Identifier of the source legal document
<code>content</code>	Chunk text encoded by Qwen and indexed by BM25
<code>title</code>	Title of the source legal document, when available
<code>legal_sectors</code>	Legal sector metadata used for domain filtering
<code>legal_type</code>	Type of legal document, such as decree or circular
<code>issuing_authority</code>	Authority that issued the document
<code>issuance_date</code>	Issuance date of the source document
<code>url</code>	Original online source, when available

The final number of chunks depends on the exact education-sector filtering and chunking run. For this reason, this report describes the dataset construction procedure and output schema rather than claiming that the education subset has the same size as the original multi-domain dataset.

2.3 Legal Sector Filtering

Because the original corpus spans many legal sectors, this project selects one target sector for controlled experimentation. Let S be the set of legal sectors in the dataset and let $s^* \in S$ be the selected target sector. In this work, the target sector is education:

$$s^* = \text{education}.$$

The filtered corpus is constructed as:

$$D_{s^*} = \{(x_i, s_i, m_i) \in D \mid s_i = s^*\}.$$

In the current implementation, education-domain documents are selected through the metadata field `legal_sectors`. The configured target is the education sector; a document is retained when the normalised sector value exactly matches this target. This keeps the experimental subset aligned with the behaviour of `src/preprocessing/chunker.py` and avoids reporting a broader keyword-based filter that is not used by the current code.

This filtering strategy has three advantages. First, it reduces the computational cost of Qwen embedding, FAISS indexing, BM25 indexing, and graph expansion. Second, it makes manual inspection of retrieval errors easier because all queries and documents belong to a coherent legal domain. Third, it keeps the experiment realistic because education regulations still contain formal legal structure, exact references, tables, appendices, and cross-document context.

2.4 Data Preprocessing

The preprocessing stage is not a simple fixed-window splitter. Instead, it is a rule-based legal-document parser designed around the structure of Vietnamese legal texts. Its purpose is to produce retrieval chunks that correspond to meaningful legal units while preserving enough metadata for retrieval, graph construction, and evaluation.

2.4.1 Input Alignment and Sector Filtering

The chunker receives two Hugging Face datasets: one containing document content and one containing metadata. The first preprocessing check ensures that both datasets have the same number of records. For each index, the content record and metadata record are paired, and their IDs are compared. If the content ID and metadata ID differ, the script logs a warning but still uses the content-side ID as the source document identifier.

Documents are filtered using `sector_matches`. The metadata field `legal_sectors` can appear as a string, list, tuple, set, or dictionary, so the chunker first normalises the value into lowercase sector strings. A document is retained only when one of these normalised sectors exactly matches the target legal sector. In the experiments, this target is the education sector. This means the implemented filter is metadata-sector based, rather than a broad keyword search over title, summary, and content.

2.4.2 Text Normalisation for Parsing

The first text-processing function is `normalize_text`. It applies Unicode NFC normalisation, converts Windows and old Mac line endings into newline characters, replaces non-breaking spaces, collapses repeated spaces and tabs inside each line, and removes excessive blank lines. Importantly, it does not flatten all newlines. Line boundaries are preserved because the parser needs individual lines to recognise part, chapter, section, article, administrative-tail, and appendix starts.

After a chunk has already been structurally identified, `flatten_chunk_text` is used before storing and embedding. This second normalisation step converts newlines and tabs into spaces, collapses repeated whitespace, and removes spaces before punctuation. Therefore, the parser uses a line-preserving representation, while the retrieval corpus stores clean one-line chunk text.

2.4.3 Legal Heading Detection

The parser scans each retained document line by line. It uses regular expressions for four main structural levels: part headings, chapter headings, section headings, and article headings. During the scan, it maintains the current values:

- `current_part`,

- `current_chapter`,
- `current_section`,
- `current_article_number`,
- `current_article_title`.

When a part, chapter, or section heading is found, any open article is flushed first, then the corresponding structural state is updated. The implementation also handles headings whose title is placed on the next line. If a part, chapter, or section line has no inline title, the variable `pending_title_level` records which level is waiting for a title. The next line is attached to that level only if `looks_like_title` decides that it resembles a heading title rather than article body text.

2.4.4 Article Chunk Construction

Article chunking is handled by the internal `flush_article` function. When a new article heading is detected, the previous article is saved, the article order counter is incremented, and a new buffer starts with the article heading line. All following non-structural lines are appended to this buffer until another structural boundary, administrative tail, appendix start, or end of document is reached.

Before saving an article chunk, the chunker performs several checks and transformations. If the raw article content contains table separators, it is passed through `clean_table_text`; otherwise it is flattened with `flatten_chunk_text`. Chunks shorter than `min_chunk_chars` are discarded. The stored article label is built from the article number and optional article title, while the chunk ID has the form:

`document_id.dieu_articleNumber_articleOrder.`

The article body used for link extraction excludes the first line of the buffer. This prevents the article heading itself from being misinterpreted as a self-reference.

2.4.5 Document Header Links

Before line-by-line parsing begins, the chunker finds the first article heading in the document. The text before this first article is treated as the document header. This header is scanned with `extract_link_to` using the scope `document_header`. The extracted header links are not duplicated into every chunk. Instead, they are merged only into the first generated article chunk of the document. This keeps cross-document legal citations from the preamble while avoiding repeated identical metadata across all article chunks.

2.4.6 Administrative Tail Handling

A major difference from basic chunking is that the script explicitly detects administrative endings. These include recipient lists, signing blocks, and role labels such as chairperson, minister, director, and deputy roles. When such a line is detected, the current article is flushed and the parser enters `inside_admin_tail`. Lines in this state are ignored so they are not attached to the last article and are not used for `link_to` extraction.

The implementation also handles the case where a recipient-list marker appears in the middle of a line. In that case, the part before the marker is preserved as the final content of the current article, the article is flushed, and the remaining administrative text is skipped. This rule prevents signature and recipient metadata from polluting the final legal provision.

2.4.7 Appendix and Table Chunking

Appendix detection is stateful. The chunker recognises explicit appendix starts, category lists, procedure sections, and appendix titles that appear after or inside administrative-tail text. If an appendix starts while an article is open, the article is flushed first. If another appendix begins while the parser is already inside an appendix, the previous appendix is flushed before starting the new one.

Appendix lines are stored with their original line boundaries because tables must be detected before flattening. The function `split_appendix_blocks` separates appendix content into consecutive text blocks and table blocks. A line is considered table-like when it contains at least two vertical-bar separators. Text blocks are flattened normally, while table blocks are cleaned with `clean_table_text`. This table cleaner removes empty separator cells, joins useful cells with semicolons, and joins rows with sentence boundaries so that table content becomes more suitable for embedding and BM25 indexing.

Long appendix or table blocks are then split by `split_long_text` with a maximum length of 3,000 characters. The splitter prefers sentence boundaries and only falls back to whitespace cuts when a single sentence or table row is too long. The resulting appendix and table chunk IDs follow separate patterns:

`document_id_phu_luc_blockOrder_partIndex` $\rightarrow$ appendix chunk, (1)

`document_id_bang_blockOrder_partIndex` $\rightarrow$ table chunk. (2)

This design keeps appendices and tables searchable as independent retrieval units instead of attaching them to the final article.

2.4.8 Legal Reference Extraction

The field `link_to` is created by `extract_link_to`. The extractor identifies several kinds of references:

- external references to legal documents by document type and document number,
- internal article references,
- clause-level or point-level references attached to an article,
- appendix references,
- amendment and repeal signals when relevant keywords occur near a referenced document.

Each extracted link stores structured fields such as relation type, target document number, article number, clause numbers, point, appendix, section, scope, and raw matched text.

Self-references to the same article or to the same document without a finer target are removed where possible. Duplicate links are also merged by key so that the final `link_to` list remains compact.

2.4.9 Output Schema and Stable Identifiers

Every saved chunk preserves both retrieval text and legal metadata. The output row contains `id`, `document_id`, `document_number`, `title`, `url`, `legal_type`, `legal_sectors`, `issuing_authority`, `issuance_date`, `signers`, `part`, `chapter`, `section`, `articles`, `content`, and `link_to`. Identifiers are normalised with `make_safe_id` so that the same IDs can be used consistently by BM25 lookup rows, FAISS metadata rows, graph source chunk IDs, and evaluation labels.

Formally, each retained document x_i is converted into structurally typed chunks:

$$x_i \rightarrow \{c_{i1}^{\text{article}}, \dots, c_{ia}^{\text{article}}, c_{i1}^{\text{appendix}}, \dots, c_{ip}^{\text{appendix}}, c_{i1}^{\text{table}}, \dots, c_{it}^{\text{table}}\}.$$

The number of chunks depends on the number of detected articles, appendices, tables, and long-block splits. Thus, the unit of retrieval is not a fixed number of tokens or characters; it is the smallest legal unit that the rule-based parser can safely extract from the source document.

Chunk types are inferred from identifier patterns:

$$\text{_dieu_} \rightarrow \text{article}, \quad (3)$$

$$\text{_phu_luc_} \rightarrow \text{appendix}, \quad (4)$$

$$\text{_bang_} \rightarrow \text{table}. \quad (5)$$

Any unmatched identifier is assigned the generic type `chunk`. In the current system, these identifiers are still useful for BM25, FAISS, lookup tables, and evaluation. The graph builder, however, focuses on article-labelled rows for hierarchy construction and uses `link_to` for document-reference edges. Appendix and table chunks therefore remain searchable retrieval units, while dedicated appendix/table graph nodes are left as future work.

Before evaluation, the system checks that BM25 lookup rows, embedding metadata rows, FAISS vectors, and graph source chunk IDs all refer to the same education-domain corpus. These consistency checks prevent stale indexes or mismatched chunk identifiers from corrupting hybrid fusion and graph expansion.

Chapter 3. Overall System Pipeline

This section summarises the complete pipeline implemented by `src/main.py` and configured by `config.json`. The application entry point is `LegalRAGApplication`. The default mode is `experiment`; in this mode, the system reuses existing preprocessed chunks and indexes unless `pipeline.rebuild_before_evaluation` is enabled.

3.1 Configured Data Flow

The current configuration uses the following main paths:

Stage	Configured output
Chunk corpus	<code>data/legal_chunks.parquet</code>
Embeddings	<code>embedding_output/embeddings.npy</code>
Embedding metadata	<code>embedding_output/embedding_metadata.parquet</code>
FAISS index	<code>faiss_output/legal_chunks.faiss</code>
BM25 index	<code>bm25_output/</code>
BM25 lookup	<code>bm25_output/bm25_lookup.parquet</code>
Graph nodes	<code>graph/nodes.parquet</code>
Graph edges	<code>graph/edges.parquet</code>
Evaluation outputs	<code>evaluation_output/</code>

The pipeline can be written as:

$$D_{raw} \rightarrow C \rightarrow (E, I_{FAISS}, I_{BM25}, G) \rightarrow R(q) \rightarrow \text{Eval}(R(q), Y_q),$$

where D_{raw} is the raw legal-document dataset, C is the chunk table, E is the embedding matrix, I_{FAISS} and I_{BM25} are the dense and lexical indexes, G is the legal graph, $R(q)$ is a ranked list for query q , and Y_q is the set of relevant chunk IDs used for evaluation.

3.2 Build Workflow

When source rebuilding is requested, the application loads the Hugging Face dataset `minhdoan17/vietnamese-legal-documents` with separate `content` and `metadata` configurations. It then calls `chunk_dataset` with target sector the education sector, minimum chunk length 50, and output path `data/legal_chunks.parquet`. The configured build steps are:

$$\text{download_models} \rightarrow \text{embed} \rightarrow \text{faiss} \rightarrow \text{bm25} \rightarrow \text{graph}.$$

The goal of this workflow is reproducibility. Every downstream structure is derived from the same chunk table, so FAISS row IDs, BM25 lookup rows, graph source chunk IDs, and evaluation labels can refer to the same stable identifiers.

3.3 Retrieval and Evaluation Workflow

For each evaluation query, the runner first computes vector candidates and BM25 candidates once:

$$C(q) = (R_{\text{vec}}(q, 250), R_{\text{BM25}}(q, 250)).$$

This cache is reused for all hybrid, graph, and hybrid-plus-graph hyperparameter trials. The final methods are evaluated at $K \in \{5, 10, 50, 100, 200\}$ and saved to `evaluation_report.json`, `method_comparison.csv`, and `query_level_results.csv`. The benefit of this design is that expensive Qwen query encoding and BM25 retrieval are not repeated for every

parameter setting; only rank fusion, graph propagation, and metric computation are repeated.

3.4 System Modes

The application supports five modes. `build` constructs artifacts, `experiment` tunes and evaluates all methods, `evaluate` aliases the full experiment, `search` retrieves for a single query, and `answer` runs retrieval followed by optional generation. In the current configuration, generation is disabled by default, so the reported results measure retrieval quality rather than answer fluency.

Chapter 4. Retrieval Methods and Mathematical Formulation

This section describes each retrieval method used by the implementation. For every method, the report states its workflow, optimisation goal, main advantage, and mathematical score.

4.1 Retrieval-Augmented Generation Principle

Workflow. Given a query q , the retriever returns an ordered list of evidence chunks:

$$R_K(q) = \{c_1, c_2, \dots, c_K\}.$$

The answer generator, when enabled, conditions on both the query and retrieved evidence:

$$\hat{a} = \arg \max_a P(a \mid q, R_K(q)).$$

Goal. The retrieval stage should place legally correct evidence near the top of $R_K(q)$ so that the generator or user sees authoritative chunks first.

Strength. This design separates factual grounding from language generation. In the current configuration, generation is disabled, so the experiment evaluates retrieval independently.

4.2 Dense Vector Retrieval

Workflow. The system loads Qwen/Qwen3-Embedding-0.6B, encodes every chunk in the `content` column, stores the resulting matrix in `embedding_output/embeddings.npy`, and builds a FAISS inner-product index. At query time, the same embedder encodes the query and FAISS returns the top 250 dense candidates.

For a text sequence x , the encoder returns token representations:

$$H = f_\theta(x) \in \mathbb{R}^{L \times d}.$$

The implementation uses attention-mask-aware mean pooling:

$$\tilde{e}(x) = \frac{\sum_{i=1}^L m_i H_i}{\max\left(1, \sum_{i=1}^L m_i\right)},$$

followed by optional ℓ_2 normalisation, enabled by `embedding.normalize=true`:

$$e(x) = \frac{\tilde{e}(x)}{\|\tilde{e}(x)\|_2}.$$

Since the FAISS metric is configured as inner product, the dense score is:

$$s_{\text{vec}}(q, c) = e(q)^\top e(c).$$

Goal. Dense retrieval tries to find chunks that are semantically close to the query, even when the query does not reuse the exact legal wording.

Strength. It is useful for paraphrased or scenario-based questions. Its weakness is that exact article numbers and document codes can be underweighted if semantic similarity dominates lexical precision.

4.3 BM25 Lexical Retrieval

Workflow. The BM25 builder tokenises the same ordered chunk corpus using `bm25s`, disables stopword removal with `stopwords=None`, and stores a lookup table in `bm25_output/bm25_lookup.pkl`. At query time, BM25 returns up to 250 lexical candidates.

BM25 scores a chunk c for query q as:

$$\text{BM25}(c, q) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, c)(k_1 + 1)}{f(t, c) + k_1 \left(1 - b + b \frac{|c|}{\text{avgdl}}\right)}.$$

Here $f(t, c)$ is the frequency of term t in chunk c , $|c|$ is chunk length, and `avgdl` is average chunk length.

Goal. BM25 maximises exact lexical overlap while controlling the effect of long chunks and repeated terms.

Strength. It works well for Vietnamese legal retrieval because document numbers, article identifiers, decree names, and official phrases are highly informative and should not be removed as stopwords.

4.4 Weighted Reciprocal Rank Fusion

Workflow. The `HybridRetriever` first obtains R_{vec} and R_{BM25} . It then calls `ReciprocalRankFusion` to combine ranks, not raw scores. The configured defaults are `semantic_weight=1.0`, `lexical_weight=1.0`, and `rrf_k=60`; tuning explores weights $\{0.5, 1.0, 1.5\}$ and $k \in \{30, 60, 90\}$.

For candidate c , the fused score is:

$$s_{\text{RRF}}(c) = \frac{w_v}{k_{\text{RRF}} + r_v(c)} + \frac{w_b}{k_{\text{RRF}} + r_b(c)},$$

where $r_v(c)$ and $r_b(c)$ are dense and BM25 ranks.

Goal. RRF aims to keep candidates that rank highly in either or both component systems without requiring BM25 scores and cosine scores to be calibrated onto the same numeric scale.

Strength. It is robust because ranks are easier to combine than heterogeneous raw scores. A chunk found by both dense and lexical search receives evidence from both lists, while a chunk found by only one component can still survive if its rank is high.

4.5 Legal Graph Construction

Workflow. The graph builder reads `data/legal_chunks.parquet` and creates stable nodes for documents and legal hierarchy units. The current source implements the hierarchy:

DOCUMENT $\rightarrow$ PART $\rightarrow$ CHAPTER $\rightarrow$ SECTION $\rightarrow$ ARTICLE.

The current builder constructs hierarchy nodes from article-labelled rows and writes document-reference edges derived from `link_to`. Nodes are written to `graph/nodes.parquet`; edges are written to `graph/edges.parquet`.

Formally, the graph is:

$$G = (V, E),$$

where each edge $e = (u, v)$ has an edge type and weight w_e .

Goal. The graph represents structure that is not visible from chunk text alone: document containment, sibling chunks, appendices, tables, and extracted legal references.

Strength. It allows retrieval to move from a directly matched chunk to structurally related evidence, such as an appendix table in the same document or a referenced legal instrument.

4.6 Graph-Only Expansion

Workflow. Graph retrieval starts from the tuned hybrid ranking as seed chunks. Each seed chunk is mapped to graph nodes through `chunk_to_nodes`. The expander performs breadth-first traversal using the configured direction `both`, maximum hops $H \in \{1, 2\}$, and hop decay $\gamma \in \{0.5, 0.65, 0.8\}$ during tuning.

For a chunk c reached through path p , the propagated graph score is:

$$s_g(c) = \max_{p \in \mathcal{P}(c)} s_{\text{seed}}(p_0) \prod_{e \in p} w_e \gamma.$$

The implementation keeps the strongest path rather than summing all paths, reducing over-reward for highly connected documents.

Goal. Graph-only retrieval tests whether structural neighbourhood alone can recover relevant legal chunks after using hybrid retrieval only to choose seeds.

Strength. It is helpful for multi-hop or context-heavy legal questions. Its risk is structural noise: sibling chunks may be related by document structure but not relevant to the query.

4.7 Hybrid Plus Graph Retrieval

Workflow. Hybrid-plus-graph first computes the RRF hybrid ranking, then expands graph neighbours from that ranking, then combines retrieval and graph scores. The default configuration uses `retrieval_weight=0.8` and `graph_weight=0.2`; tuning evaluates graph weights $\{0.1, 0.2, 0.3\}$.

The final score is:

$$s_{\text{final}}(c) = \lambda_r s_{\text{retrieval}}(c) + \lambda_g s_{\text{graph}}(c),$$

with:

$$\lambda_r + \lambda_g = 1.$$

Goal. This method preserves direct semantic and lexical evidence while allowing graph structure to promote related chunks.

Strength. It is the most complete retrieval method in the current system because it combines exact matching, semantic matching, and legal-structure propagation. It is especially useful at deeper cutoffs where graph expansion can recover additional relevant chunks.

4.8 Evaluation Objective

Workflow. The evaluator caches BM25 and vector candidates once per query, tunes hybrid parameters, tunes graph parameters using the best hybrid seeds, tunes hybrid-plus-graph, and finally evaluates BM25, vector, graph, hybrid, and hybrid-plus-graph.

For a configuration θ , the tuning objective is:

$$J(\theta) = \frac{1}{2|\mathcal{K}|} \sum_{K \in \mathcal{K}} (\text{Recall@K}_\theta + \text{MRR@K}_\theta),$$

where:

$$\mathcal{K} = \{5, 10, 50, 100, 200\}.$$

Goal. The objective balances coverage and rank quality. Recall rewards retrieving labelled evidence somewhere in the candidate list, while MRR rewards placing the first relevant chunk near the top.

Strength. This objective avoids selecting a configuration that only performs well at very shallow ranks or only performs well at deep recall.

Chapter 5. Data Preprocessing Pipeline

This chapter describes the implemented preprocessing code in `src/preprocessing/chunker.py`. The file does more than basic string cleaning: it converts raw Vietnamese legal documents into article, appendix, and table retrieval chunks while preserving metadata and extracting legal references. This stage is important because every later component—BM25, FAISS, graph construction, and evaluation—depends on the chunk identifiers and metadata produced here.

5.1 Dataset Pairing and Input Validation

The chunker receives two dataset objects: one for content and one for metadata. Both may be a `Dataset` or a `DatasetDict`, so the helper `get_data_split` selects the usable split before processing. The implementation first checks that the two datasets have the same number of rows. If they do not match, it raises an explicit error because content rows and metadata rows would no longer describe the same legal documents.

For each index, the chunker reads the content item and metadata item together. The source document ID is taken from the content row when available, otherwise from metadata, otherwise from the row index. If the content ID and metadata ID disagree, the script logs a warning. This validation does not stop processing, but it exposes possible dataset alignment problems before indexes are built.

5.2 Legal-Sector Filtering

The target sector is passed as `target_legal_sector`. Before chunking, the code checks whether at least one metadata row matches this sector. If no match exists, the function logs an error and returns an empty result. During the main loop, only matching documents are retained; non-matching documents are counted as skipped.

Sector matching is handled by `sector_matches`. Since `legal_sectors` may be stored as a string, list, tuple, set, or dictionary, the value is normalised before comparison. This makes the filter robust to several dataset serialisation formats. The comparison itself is exact after normalisation, so the current implementation is metadata-sector based rather than a loose keyword search over the whole document text.

5.3 Two-Level Text Normalisation

The preprocessing code uses two different text normalisation functions because parsing and retrieval have different requirements. The first function, `normalize_text`, prepares the raw document for structural parsing. It applies Unicode NFC normalisation, converts all line-ending styles to newline characters, replaces non-breaking spaces, collapses repeated spaces and tabs within each line, and removes excessive blank lines. Newlines are intentionally preserved because legal headings such as parts, chapters, sections, and articles are detected line by line.

After the parser has already decided which lines belong to a chunk, the second function, `flatten_chunk_text`, prepares the chunk for storage and embedding. It converts line breaks and tabs into spaces, collapses repeated whitespace, and removes spaces before punctuation. Therefore, the pipeline keeps structure while parsing but stores a clean one-line text representation for BM25 and vector retrieval.

5.4 Document Header Processing

Before the main line scan, the chunker searches for the first article heading. Text before the first article is treated as the document header. This header often contains legal bases, citations to laws or decrees, and issuing context. The header is passed to `extract_link_to` with scope `document_header`.

Header links are not copied into every chunk. Instead, they are merged only into the first generated article chunk of the document. This design keeps important preamble citations while avoiding repeated identical `link_to` metadata in every article chunk.

5.5 Line-by-Line Legal Structure Parser

The main parser scans the normalised document line by line. It uses regular expressions to detect four structural levels:

- part headings,
- chapter headings,
- section headings,
- article headings.

The parser maintains the current structural state using `current_part`, `current_chapter`, `current_section`, `current_article_number`, and `current_article_title`. When a new part, chapter, or section begins, any open article is flushed first, then the state is updated. When a new article begins, the previous article is saved and a new article buffer starts.

The implementation also handles headings whose title appears on the following line. If a part, chapter, or section heading has no inline title, `pending_title_level` records which structural level is waiting for a title. The next non-structural line is attached to that level only when `looks_like_title` judges it to be a heading-style title. This avoids accidentally treating normal article content as a chapter or section title.

5.6 Article Chunk Creation

Article chunking is implemented through the internal `flush_article` function. An article buffer starts at an article heading and accumulates all following lines until a new structural boundary, administrative tail, appendix start, or document end is reached. If no current article is open, flushing simply clears the buffer.

Before saving, the function converts the article buffer into retrieval text. If the raw content contains table separators, `clean_table_text` is used; otherwise `flatten_chunk_text` is used. Very short chunks are discarded using `min_chunk_chars`. The article label stored in the `articles` field combines the article number and optional article title.

The article body used for reference extraction deliberately excludes the first line of the article buffer. This prevents the article heading itself from being interpreted as a self-reference. The generated article chunk ID follows the pattern:

```
safeDocumentId_dieu_safeArticleNumber_articleOrder.
```

The ordered suffix makes IDs stable even when legal documents contain repeated or unusual article numbers.

5.7 Administrative Tail Removal

Vietnamese legal documents often end with recipient lists, signature blocks, and role titles. If these lines are appended to the last legal article, retrieval quality becomes worse because administrative text is mixed with normative content. The chunker avoids this using two rules.

First, `ADMIN_TAIL_PATTERN` detects administrative-tail lines such as recipient-list markers and signing roles. When such a line is found, the current article is flushed and the parser enters `inside_admin_tail`. Lines in this state are skipped and are not used for `link_to` extraction.

Second, `ADMIN_INLINE_PATTERN` handles the case where the recipient marker occurs in the middle of a line. The prefix before the marker is preserved as the final content of the current article, then the article is flushed, and the remaining administrative tail is ignored unless an appendix title is detected inside it.

5.8 Appendix Detection and State Handling

Appendices are handled as a separate parser state. The code recognises explicit appendix starts, category-list starts, procedure-list starts, and inline appendix titles. If an appendix begins while an article is open, the article is flushed before the appendix buffer starts. If a second appendix starts while the parser is already inside an appendix, the previous appendix is flushed first.

The parser can also recover appendices that appear after the signature or recipient section. While `inside_admin_tail` is true, ordinary administrative lines are ignored, but appendix-start patterns are still checked. This is important because many Vietnamese documents place appendices after the signature block.

5.9 Table-Aware Appendix Splitting

Appendix content is stored line by line until it is flushed. The function `split_appendix_blocks` then separates the appendix into consecutive text blocks and table blocks. A line is treated as table-like when it contains at least two vertical-bar separators. Consecutive lines of the same kind are grouped together.

Text blocks are passed through `flatten_chunk_text`. Table blocks are passed through `clean_table_text`. The table cleaner removes empty separator cells, discards rows made only of separator characters, joins cells with semicolons, and joins rows with sentence

boundaries. This converts Markdown-like tables into text that is easier for BM25 and embedding models to index.

Very long appendix or table blocks are split by `split_long_text` with a maximum length of 3,000 characters. The splitter prefers sentence boundaries and falls back to whitespace cuts only when one sentence or table row exceeds the limit. The resulting IDs distinguish appendix text from table chunks:

`safeDocumentId.phu.luc.blockOrder.partIndex` → appendix text, (6)

`safeDocumentId.bang.blockOrder.partIndex` → table text. (7)

This is why appendices and tables remain searchable in BM25 and FAISS as independent chunks, even though the current graph builder focuses on the article hierarchy and document-level reference edges.

5.10 Legal Reference and Relationship Extraction

The chunker extracts legal links using `extract_link_to`. The extractor recognises external document references, internal article references, clause-level references, point-level references, and appendix references. It also uses keyword patterns to mark amendment and repeal relationships when terms such as amendment, replacement, repeal, or expiry occur near a referenced document.

Each link is stored as a structured object rather than as raw text only. Important fields include relation type, scope, target document number, document type, article number, clause numbers, point, appendix, section, raw matched text, and whether the target is internal or external. The scope distinguishes links extracted from the document header, an article, or an appendix/table chunk.

The code also removes weak self-references where possible. For example, the current article heading is not scanned as body text, and links that point only to the same document without a more specific target are ignored. Duplicate links are merged using `merge_links`, keeping the `link_to` field compact.

5.11 Output Chunk Schema

Every generated chunk is stored with both retrieval text and source metadata. The output dictionary contains:

- `id`, `document_id`, and `document_number`,
- title, URL, legal type, legal sectors, issuing authority, issuance date, and signers,
- current part, chapter, and section,
- the article or appendix/table label in `articles`,
- cleaned chunk `content`,
- structured legal references in `link_to`.

The final chunk table is written to the configured parquet output path. These stable chunk IDs are the common key used later by BM25 lookup rows, FAISS metadata rows, graph source chunk IDs, and evaluation labels.

5.12 Relation to Index Builders and Graph Mapping

The embedding and BM25 builders consume the preprocessed `content` field rather than raw legal documents. Missing content values are converted to empty strings, every value is cast to string, and the Qwen tokenizer truncates embedding inputs to 1,024 tokens. BM25 uses the same ordered corpus and disables stopword removal by setting `stopwords=None`, because Vietnamese legal retrieval often depends on numbers, abbreviations, and short official phrases.

The graph layer relies on the chunk identifiers produced in this chapter. In the current builder, article-labelled rows are grouped into article nodes and their chunk IDs are stored in `source_chunk_ids`. During graph loading, these values are parsed from JSON strings, Python collections, or fallback comma-separated strings before constructing the mapping:

chunk ID $\rightarrow$ graph node IDs.

This final mapping prevents the common bug of treating a serialized string as an iterable of individual characters, and it ensures that article retrieval chunks can be connected back to one or more legal graph nodes.

Chapter 6. Dense Passage Embedding and Vector Store Construction

6.1 Embedding Model

The current encoder is:

Qwen/Qwen3-Embedding-0.6B.

The system first checks the configured local model directory and falls back to the Hugging Face model identifier when the local copy is unavailable.

6.2 Device and Precision

The model selects CUDA automatically when available. Float16 is used only on CUDA; CPU inference falls back to float32 for compatibility.

6.3 Batch Encoding

For corpus texts $\{x_i\}_{i=1}^N$, vectors are computed batch by batch and concatenated:

$$E = \begin{bmatrix} e(x_1)^\top \\ e(x_2)^\top \\ \vdots \\ e(x_N)^\top \end{bmatrix} \in \mathbb{R}^{N \times d}.$$

The matrix is saved as float32 in:

`embedding_output/embeddings.npy`.

6.4 FAISS Index

The configured similarity metric is inner product. The FAISS index is saved at:

`faiss_output/legal_chunks.faiss`.

At loading time, the system verifies:

$$N_{\text{FAISS}} = N_{\text{metadata}}.$$

If the counts differ, retrieval is stopped with a descriptive exception.

6.5 BM25 Index Construction

The BM25 builder indexes the same ordered corpus used by the embedding builder. It uses the `bm25s` library to tokenise all chunk texts and save a persistent BM25 index. The full chunk table is also written to `bm25_output/bm25_lookup.parquet`. During retrieval, BM25 returns row indices; the lookup table converts those indices back to chunk identifiers, chunk text, and metadata.

This design is important because vector search and lexical search use different storage formats. FAISS returns vector positions, whereas BM25 returns lexical-document positions. The metadata and lookup tables therefore act as the common identifier layer that allows rank fusion to combine both result lists safely.

Chapter 7. Legal Knowledge Graph Construction

The graph builder is implemented in `src/graph_rerank/graph_builder.py`. It reads the preprocessed chunk table and builds two complementary graph structures: a legal hierarchy inside each document and document-level reference edges derived from `link_to`.

7.1 Node Types

The current graph contains the following node types:

- `document` nodes,
- `part` nodes,
- `chapter` nodes,

- `section` nodes,
- `article` nodes,
- `external_document` nodes for unresolved or out-of-corpus references.

Node IDs are generated with stable SHA-1 based identifiers:

$$\text{node_id} = \text{prefix} : \text{SHA1}(\text{value})_{1:20}.$$

This makes graph rebuilds deterministic as long as the input chunk metadata is unchanged.

7.2 Hierarchy Edges

For article-labelled rows, the builder validates part, chapter, section, and article labels with regular expressions. Missing part, chapter, or section levels are represented by virtual nodes, so every article can still be placed into a complete hierarchy:

$$\text{document} \rightarrow \text{part} \rightarrow \text{chapter} \rightarrow \text{section} \rightarrow \text{article}.$$

The corresponding edge types are `CONTAINS_PART`, `CONTAINS_CHAPTER`, `CONTAINS_SECTION`, and `CONTAINS_ARTICLE`. If several chunks refer to the same article node, their contents and source chunk IDs are accumulated into that node instead of creating duplicate article nodes.

7.3 Document Reference Edges

The builder also iterates over the `link_to` column. For each extracted link with a target document number, it resolves the target to an existing document node when possible; otherwise it creates an `external_document` node. It then creates a `DOCUMENT_LINK` edge. Repeated references between the same source, target, and relationship are accumulated by increasing `reference_count`.

The edge table stores:

`source`, `target`, `edge_type`, `reference_count`, and relation metadata.

This means the current graph captures both structural containment and cross-document citation evidence.

7.4 Chunk-to-Node Mapping

Article graph nodes store source chunk IDs in `source_chunk_ids`. During graph loading, these values are parsed into lists before constructing:

$$\text{chunk ID} \rightarrow \text{graph node IDs}.$$

The loader accepts JSON arrays, Python collections, and fallback comma-separated strings. This prevents the common implementation error of iterating through a serialized string character by character.

7.5 Graph Traversal and Propagation

The configured traversal direction is **both**, so expansion can follow hierarchy edges upward or downward and can follow document-reference edges. Graph expansion starts from retrieved seed chunks, maps them to graph nodes, and performs breadth-first traversal up to the configured hop count.

For a candidate chunk c reached through path p , the propagated score is:

$$s_g(c) = \max_{p \in \mathcal{P}(c)} s_{\text{seed}}(p_0) \prod_{e \in p} w_e \gamma.$$

The maximum keeps the strongest path explanation for a candidate. The product discounts unreliable or long paths through edge weights w_e and hop decay γ .

Chapter 8. Retrieval Runtime Architecture

8.1 BM25 Retrieval

The BM25 index and lookup table are loaded once. For a query, the system tokenises the text and retrieves up to 250 candidates. Each result is converted to a common `RetrievalResult` object.

8.2 Vector Retrieval

The query is encoded once and searched against FAISS. Up to 250 dense candidates are returned, together with scores and metadata.

8.3 Component Caching

During evaluation, BM25 and vector results are computed once for each query:

$$C(q) = (R_{\text{vec}}(q), R_{\text{BM25}}(q)).$$

All subsequent hyperparameter combinations reuse this cache. Therefore, tuning RRF and graph parameters does not repeatedly run Qwen query encoding or BM25 retrieval.

8.4 Application Entry Point

The `LegalRAGApplication` class provides five execution modes:

- **build**: download models and construct embeddings, FAISS, BM25, and graph,
- **experiment**: tune and evaluate all methods,
- **evaluate**: alias of the complete experiment,
- **search**: run one selected retrieval method for a query,
- **answer**: retrieve evidence and invoke the answer generator.

The default mode is **experiment**.

Chapter 9. Answer Generation

9.1 Generation Model

The optional answer generator is configured with:

`Qwen/Qwen3-0.6B.`

Generation is disabled by default during retrieval experiments so that evaluation measures retrieval quality independently from generative behaviour.

The generator checks `generation.enabled` before loading the model. If generation is disabled, it raises an explicit runtime error rather than silently producing an answer without a configured generator. This keeps the experiment mode retrieval-only and prevents answer quality from being confused with retrieval quality.

9.2 Grounded Generation

When enabled, the generator receives the query and retrieved chunks. Retrieved identifiers should be preserved in the prompt so that the answer can cite its supporting evidence.

9.3 Separation of Concerns

The system separates:

1. retrieval evaluation,
2. hyperparameter tuning,
3. answer generation.

This avoids attributing generation errors to retrieval and allows retrieval improvements to be measured directly.

Chapter 10. Evaluation Framework

10.1 Evaluation Queries

The evaluation dataset is loaded from:

`ground_truth/all_queries.jsonl.`

Each record may be JSONL, JSON, or CSV. The loader accepts `query`, `question`, or `text` as the query field and accepts `relevant_chunk_ids`, `relevant_ids`, `ground_truth`, or `answers` as relevance labels. Each parsed record contains:

- a query ID,
- the query text,
- one or more relevant chunk IDs.

The experiment uses 4,000 evaluation queries. Query order is shuffled when enabled in the configuration. In the current experimental design, the selected queries are used both for hyperparameter tuning and for final method comparison.

10.2 Evaluation Cutoffs

All methods are evaluated at:

$$K \in \{5, 10, 50, 100, 200\}.$$

A cutoff K means that only the first K retrieved chunks are considered correct or incorrect for that metric. Small cutoffs such as 5 and 10 measure whether the system is useful for a user or generator that reads only the top evidence. Larger cutoffs such as 100 and 200 measure candidate-generation ability: they show whether the method can find relevant legal chunks somewhere in a broad candidate pool, even if the final order is not ideal. The maximum cutoff is also used to define a missed query.

10.3 Hit@K

For query q_i , relevant set G_i , and retrieved ranking $R_K(q_i)$:

$$\text{Hit@K}(q_i) = \begin{cases} 1, & R_K(q_i) \cap G_i \neq \emptyset, \\ 0, & \text{otherwise.} \end{cases}$$

The reported method-level value is:

$$\text{Hit@K} = \frac{1}{N} \sum_{i=1}^N \text{Hit@K}(q_i).$$

Hit@K is a binary success measure for each query. It does not care how many relevant chunks are found; it only checks whether at least one labelled relevant chunk appears in the top K . Therefore, Hit@K is easy to interpret as the proportion of queries for which the retriever gives the user at least one useful piece of evidence. Its limitation is that it treats a query with one relevant chunk found the same as a query with many relevant chunks found.

10.4 Recall@K

When a query may contain multiple relevant chunks:

$$\text{Recall@K}(q_i) = \frac{|R_K(q_i) \cap G_i|}{|G_i|}.$$

The macro average is:

$$\text{Recall@K} = \frac{1}{N} \sum_{i=1}^N \text{Recall@K}(q_i).$$

Recall@K measures coverage of all labelled evidence. If a query has four relevant chunks and the top- K ranking contains two of them, its Recall@K is $2/4 = 0.5$. The report uses macro averaging, so each query contributes equally regardless of how many relevant chunks it has. This is appropriate for legal retrieval because a query with many relevant provisions should not dominate the aggregate metric only because it has more labels.

10.5 MRR@K

Let r_i be the rank of the first relevant chunk. Then:

$$\text{RR@K}(q_i) = \begin{cases} \frac{1}{r_i}, & r_i \leq K, \\ 0, & \text{otherwise.} \end{cases}$$

Mean Reciprocal Rank is:

$$\text{MRR@K} = \frac{1}{N} \sum_{i=1}^N \text{RR@K}(q_i).$$

MRR@K focuses on the position of the first relevant chunk. A relevant chunk at rank 1 gives reciprocal rank 1, at rank 2 gives 0.5, at rank 10 gives 0.1, and no relevant chunk in the top K gives 0. This makes MRR sensitive to top-rank ordering, which is important when only a few retrieved chunks can be shown to the user or inserted into a generation prompt. The implementation uses the standard name MRR rather than MMR.

10.6 Miss Count

A query is counted as a miss when:

$$\text{Hit@200}(q_i) = 0.$$

This definition separates complete retrieval failure from poor ranking. If a relevant chunk appears at rank 150, the query is not a miss, although its MRR at shallow cutoffs will be low. If no relevant chunk appears even in the top 200, the retriever has failed to generate a usable candidate for that query, and later reranking or generation stages cannot recover the missing evidence.

Therefore:

$$\text{miss} = \sum_{i=1}^N \mathbb{I}[R_{200}(q_i) \cap G_i = \emptyset].$$

10.7 Graph Diagnostic Counts

For graph-based methods, the evaluator additionally records:

- `mapped_zero`: no retrieved seed chunk could be mapped to a graph node,
- `graph_zero`: graph traversal returned no graph candidate.

These counts distinguish corpus-retrieval failures from graph-coverage or graph-connectivity failures.

10.8 Query-Level Diagnostics

For each method and query, the evaluator stores the sorted relevant chunk IDs, the ranked chunk IDs returned by the method, the miss flag, and all Hit@K, Recall@K, and MRR@K values. These rows are written to `query_level_results.csv`. The file is useful for manual error analysis because it shows whether a method failed completely, retrieved the relevant chunk only at a deep rank, or retrieved a sibling chunk that might indicate an incomplete relevance label.

Chapter 11. Hyperparameter Tuning

11.1 Hybrid Search Grid

The hybrid grid is:

$$w_v \in \{0.5, 1.0, 1.5\}, \tag{8}$$

$$w_b \in \{0.5, 1.0, 1.5\}, \tag{9}$$

$$k_{\text{RRF}} \in \{30, 60, 90\}. \tag{10}$$

This produces:

$$3 \times 3 \times 3 = 27$$

hybrid configurations.

The grid search treats w_v , w_b , and k_{RRF} as discrete design choices. Increasing w_v gives more influence to semantic similarity, which may help paraphrased questions. Increasing w_b gives more influence to exact lexical overlap, which is often important for legal identifiers and official terms. The three values of k_{RRF} test whether the system should strongly favour top-ranked candidates or use a flatter fusion curve.

11.2 Graph Grid

The graph-only tuning grid is:

$$H \in \{1, 2\}, \tag{11}$$

$$\gamma \in \{0.50, 0.65, 0.80\}. \quad (12)$$

The tuned hybrid parameters are reused as graph seeds, while graph traversal depth and decay are varied.

The parameter H is the maximum number of graph hops. With $H = 1$, expansion mainly reaches immediate neighbours such as a parent section or document-reference neighbour. With $H = 2$, traversal can move through hierarchy nodes to related article nodes or through document-level links. The decay γ controls how much trust is retained after each hop: lower values make expansion conservative, while higher values allow farther graph neighbours to keep more score.

11.3 Hybrid-Graph Grid

The hybrid-plus-graph grid is:

$$\lambda_g \in \{0.1, 0.2, 0.3\}, \quad (13)$$

$$H \in \{1, 2\}, \quad (14)$$

$$\gamma \in \{0.50, 0.65, 0.80\}. \quad (15)$$

The retrieval weight is constrained by:

$$\lambda_r = 1 - \lambda_g.$$

This grid tests graph expansion as an additive signal rather than a replacement for retrieval. When $\lambda_g = 0.1$, the final ranking remains mostly retrieval-driven. When $\lambda_g = 0.3$, graph structure has a stronger opportunity to promote chunks that are structurally related to the retrieved seeds. Values above this range were not used because legal documents can contain many sibling chunks, and overly strong graph propagation may push loosely related chunks above directly matched evidence.

11.4 Selection Objective

For a method configuration θ , the tuning objective is the average of Recall@K and MRR@K over every requested cutoff:

$$J(\theta) = \frac{1}{2|\mathcal{K}|} \sum_{K \in \mathcal{K}} (\text{Recall@K}_\theta + \text{MRR@K}_\theta),$$

where:

$$\mathcal{K} = \{5, 10, 50, 100, 200\}.$$

The configuration with maximum $J(\theta)$ is selected. The factor $2|\mathcal{K}|$ normalises the sum because each cutoff contributes two metrics: Recall@K and MRR@K. As a result, $J(\theta)$

remains on an interpretable scale between 0 and 1 when the component metrics are between 0 and 1.

This objective balances candidate coverage and ranking quality. Recall rewards methods that retrieve all labelled relevant chunks somewhere in the top- K list, while MRR rewards methods that place the first relevant chunk near the top. Averaging over five cutoffs prevents the tuner from optimising only shallow ranking or only deep recall. For example, a configuration that has excellent Recall@200 but poor MRR@5 can find relevant evidence eventually, but it may still be bad for answer generation because the best evidence is buried too deeply. Conversely, a configuration with strong MRR@5 but weak Recall@100 may answer simple single-evidence questions well but miss multi-evidence legal questions.

11.5 Number of Tuning Trials

The configured grids produce 27 hybrid trials, 6 graph-only trials, and 18 hybrid-plus-graph trials. Because the evaluator caches BM25 and vector candidates once per query, the expensive embedding and lexical retrieval steps are not repeated for each of these 51 tuning configurations. Only rank fusion, graph propagation, and metric computation are repeated.

11.6 Tuning Procedure

1. Load the configured evaluation queries.
2. Load FAISS, BM25, graph nodes, and graph edges.
3. Compute and cache vector and BM25 rankings for each query.
4. Evaluate every hybrid configuration.
5. Select the best hybrid configuration.
6. Tune graph-only parameters using the selected hybrid seed ranking.
7. Tune hybrid-plus-graph weights and graph parameters.
8. Evaluate all five final methods.
9. Print and save the comparison table.

Chapter 12. System Implementation

12.1 Class-Based Architecture

The revised source tree is organised as follows:

```
1 src/  
2 |-- main.py  
3 |-- core/  
4 |   '-- config.py  
5 |-- models/  
6 |   |-- model_downloader.py  
7 |   |-- embedder.py  
8 |   |-- index_builders.py
```

```

9 |     '-- answer_generator.py
10 |-- retrieval/
11 |     |-- schemas.py
12 |     |-- rrf.py
13 |     '-- hybrid_retriever.py
14 |-- graph_rerank/
15 |     |-- graph_builder.py
16 |     |-- graph_expander.py
17 |     '-- graph_search_pipeline.py
18 |-- evaluation_metrics/
19 |     |-- dataset.py
20 |     |-- metrics.py
21 |     '-- runner.py
22 |-- preprocessing/
23 |     '-- chunker.py
24 '-- utils/
25     '-- logger.py

```

Each Python module defines reusable classes rather than executing work at import time.

12.2 Module Responsibilities

The main responsibilities are separated as follows:

- `core.config.AppConfig` loads nested configuration values and resolves relative paths from the project root.
- `models.embedder.QwenEmbeddingModel` loads the Qwen embedding model and encodes batches of text.
- `models.index_builders` builds persistent FAISS and BM25 indexes.
- `retrieval.hybrid_retriever.HybridRetriever` exposes BM25, vector, and weighted RRF retrieval.
- `graph_rerank.GraphSearchPipeline` coordinates graph, hybrid, and hybrid-plus-graph methods.
- `evaluation_metrics.EvaluationRunner` performs caching, tuning, metric computation, and output writing.

This separation makes the system easier to debug: an error in embedding construction, graph loading, retrieval fusion, or evaluation output can be traced to a specific class.

12.3 Implementation Notes

The code-level implementation is intentionally separated from the conceptual pipeline described after the Dataset section. This section therefore only records the reusable modules and experiment artifacts, rather than repeating the full end-to-end workflow.

12.4 Output Files

The experiment writes:

- `evaluation_output/evaluation_report.json`,

- `evaluation_output/method_comparison.csv`,
- `evaluation_output/query_level_results.csv`.

The aggregate CSV contains one row per retrieval method. The query-level CSV contains approximately:

$$4000 \times 5 = 20,000$$

rows.

12.5 Reproducibility Controls

The configuration fixes `project.seed=42` and `evaluation.random.seed=42`. The evaluator logs progress every 25 cached queries by default, which is useful for long CPU-only runs. The flag `pipeline.rebuild.before_evaluation` controls whether indexes are rebuilt before evaluation; it is disabled by default so that experiments can reuse existing FAISS, BM25, and graph files.

Chapter 13. Results and Analysis

13.1 Quantitative Results

The results show the final performance of five retrieval methods over 4,000 evaluation queries. Instead of presenting all metrics in one wide table, the results are separated into Hit@K, Recall@K, and MRR@K tables for readability. All three tables use the same cutoffs $K \in \{5, 10, 50, 100, 200\}$.

Table 2: Hit@K results over 4,000 evaluation queries.

Method	Query	Miss	Hit@5	Hit@10	Hit@50	Hit@100	Hit@200
BM25	4000	194	0.7305	0.7865	0.8925	0.9277	0.9515
Vector	4000	2932	0.0963	0.1180	0.1867	0.2218	0.2670
Graph	4000	1834	0.5110	0.5230	0.5345	0.5390	0.5415
Hybrid	4000	218	0.7258	0.7857	0.8925	0.9223	0.9455
Hybrid + Graph	4000	218	0.7242	0.7855	0.8928	0.9220	0.9455

Table 3: Recall@K results over 4,000 evaluation queries.

Method	Query	Miss	Recall@5	Recall@10	Recall@50	Recall@100	Recall@200
BM25	4000	194	0.5373	0.6140	0.7652	0.8240	0.8690
Vector	4000	2932	0.0512	0.0647	0.1081	0.1305	0.1578
Graph	4000	1834	0.3835	0.4234	0.5053	0.5116	0.5162
Hybrid	4000	218	0.5225	0.6111	0.7663	0.8170	0.8600
Hybrid + Graph	4000	218	0.5213	0.6090	0.7762	0.8300	0.8888

Table 4: MRR@K results over 4,000 evaluation queries.

Method	Query	Miss	MRR@5	MRR@10	MRR@50	MRR@100	MRR@200
BM25	4000	194	0.6258	0.6334	0.6388	0.6393	0.6395
Vector	4000	2932	0.0660	0.0689	0.0722	0.0727	0.0730
Graph	4000	1834	0.4324	0.4340	0.4346	0.4347	0.4347
Hybrid	4000	218	0.5677	0.5758	0.5812	0.5817	0.5819
Hybrid + Graph	4000	218	0.5657	0.5740	0.5794	0.5799	0.5801

The miss column counts queries with no relevant chunk in the top 200 results. BM25 obtains the best shallow ranking performance, with Hit@5 of 0.7305 and MRR@5 of 0.6258. Hybrid retrieval is close to BM25 in Hit@K and Recall@K, but its MRR is lower, indicating that fusion improves or preserves candidate coverage but may move the first relevant result slightly lower in the top ranks. Vector-only retrieval performs poorly in this experiment, suggesting that Qwen3 embeddings without domain adaptation are not sufficient for exact Vietnamese legal retrieval in the education subset. Graph-only retrieval improves over vector retrieval but remains below BM25 and hybrid retrieval.

13.2 Best Hyperparameters

The best hybrid configuration selected by the experiment is:

`semantic_weight=0.5, lexical_weight=1.5, rrf_k=30.`

This confirms that lexical matching is more important than dense similarity for the current benchmark. The best hybrid-plus-graph configuration is:

`retrieval_weight=0.7, graph_weight=0.3, max_hops=2, hop_decay=0.8.`

Among the hybrid variants, hybrid-plus-graph achieves a slightly higher objective score, 0.6504, than hybrid alone, 0.6465. BM25 still has the strongest overall objective in the final table, 0.6786. The main benefit of graph expansion appears at deeper cutoffs: Recall@200 rises from 0.8600 for hybrid to 0.8888 for hybrid-plus-graph. This means graph expansion helps recover additional relevant chunks at deeper ranks, even though it does not improve top-rank MRR.

13.3 Method Interpretation

The final results should be interpreted as follows:

- **BM25** measures exact lexical matching.
- **Vector** measures semantic retrieval quality.
- **Hybrid** tests whether lexical and semantic signals are complementary.
- **Graph** tests whether document-level structural expansion from hybrid seeds can retrieve relevant neighbours.
- **Hybrid + Graph** tests whether graph evidence improves a strong fused baseline.

13.4 Shallow Versus Deep Cutoffs

Hit@5, Recall@5, and MRR@5 primarily measure whether the system places relevant chunks at the top. Recall@100 and Recall@200 measure candidate-generation coverage. A method may achieve high Recall@200 while having low MRR@10, indicating that the relevant chunk is found but ranked too deeply.

13.5 Miss Analysis

A high miss count means the relevant chunks are absent even at rank 200. Such failures are likely caused by:

- incomplete relevance labels,
- query and chunk terminology mismatch,
- overly long or noisy chunks,
- legal identifiers not represented consistently,
- insufficient semantic specialisation of the embedding model.

13.6 Graph Error Analysis

Graph-based methods should be analysed jointly with `mapped_zero` and `graph_zero`. In the final run, both graph and hybrid-plus-graph have 439 mapped-zero cases and 439 graph-zero cases:

- A high `mapped_zero` count indicates missing chunk-to-node mapping.
- A high `graph_zero` count with low `mapped_zero` indicates sparse or unusable graph connectivity.
- Low graph diagnostic counts but poor ranking metrics indicate graph-induced noise rather than graph-loading failure.

13.7 Hyperparameter Interpretation

The selected semantic and lexical weights reveal the relative importance of dense and exact matching. A lower RRF constant increases the influence of top-ranked items, whereas a larger constant makes the fusion flatter. In graph tuning, a high graph weight or two-hop traversal may improve multi-hop questions but also introduce unrelated chunks from the same document or neighbouring documents.

13.8 Failure Mode Examples

Several failure modes can be diagnosed from the output files:

- High BM25 performance but low vector performance suggests that queries depend on exact document numbers, article numbers, or legal phrases.
- High vector performance but low BM25 performance suggests paraphrased or scenario-based questions with little lexical overlap.

- Hybrid outperforming both components indicates complementary lexical and semantic evidence.
- Graph improving Recall@100 or Recall@200 but reducing MRR@5 indicates that expansion finds relevant neighbours but may add ranking noise.
- Large `mapped_zero` or `graph_zero` values indicate graph mapping or connectivity problems rather than weak retrieval scores.

Chapter 14. Limitations

14.1 Hyperparameter Tuning and Evaluation on the Same Queries

The current implementation tunes and evaluates configurations on the same set of 4,000 queries. This is useful for method comparison during development, but the reported result may be optimistic. A stronger experimental design should split queries into a tuning set and a held-out test set.

14.2 Graph Granularity

The current graph maps document, part, chapter, section, and article nodes, but does not yet represent clause-level nodes or dedicated appendix/table graph nodes. A more expressive graph should include:

document → part → chapter → section → article → clause or chunk.

14.3 Limited Relation Semantics

Containment edges are available for every chunk, but cross-document legal relations such as amendment, replacement, guidance, and repeal require accurate citation extraction and resolution.

14.4 No Learned Reranker

The current comparison does not include a cross-encoder reranker. RRF and graph propagation operate mainly on ranks and structure, without jointly encoding the query and candidate passage.

14.5 Synthetic or Automatically Constructed Queries

If evaluation queries are generated automatically, they may not reflect the ambiguity, incompleteness, and wording used by real legal practitioners or citizens.

14.6 Computational Cost

Running Qwen3 embedding and retrieving 250 vector candidates for 4,000 queries may be expensive on CPU. Graph tuning further increases computation, although component caching substantially reduces repeated work.

Chapter 15. Further Improvements

15.1 Separate Tuning and Test Sets

The 4,000-query dataset should be divided into:

$$Q_{\text{train}}, Q_{\text{validation}}, Q_{\text{test}},$$

for example 60%/20%/20%. Hyperparameters should be selected only on validation queries, and final metrics should be reported once on the held-out test set.

15.2 Cross-Encoder Reranking

A multilingual or Vietnamese legal cross-encoder can rerank the top 30–100 hybrid candidates. This is likely to improve shallow-cutoff metrics without changing candidate recall.

15.3 Domain-Adaptive Fine-Tuning

Qwen3-Embedding can be fine-tuned with contrastive learning using:

- positive query–chunk pairs,
- BM25 hard negatives,
- vector hard negatives,
- graph-neighbour hard negatives.

15.4 Relation-Aware Graph Scoring

Different legal relations should use separate weights:

$$w_{\text{replaces}}, w_{\text{amends}}, w_{\text{guides}}, w_{\text{cites}}, w_{\text{contains}}.$$

These values may be tuned or learned from labelled query–candidate data.

15.5 Query-Adaptive Graph Expansion

Graph expansion should be disabled for simple single-hop queries and enabled only for queries predicted to require references, amendments, or multiple legal instruments.

15.6 Graph Pruning

Before adding a neighbour, the system can require:

$$\cos(e(q), e(c_{\text{neighbour}})) \geq \tau.$$

This combines graph connectivity with semantic relevance and reduces structural noise.

15.7 Approximate Vector Indexing

For larger corpora, FAISS HNSW or IVF indexes can reduce latency while preserving most retrieval quality. Exact and approximate indexes should be compared using the same evaluation benchmark.

15.8 Real-User Evaluation

A future benchmark should include queries from legal professionals, students, public-service users, and administrative procedures. Relevance should be judged by more than one annotator.

Chapter 16. Author Contributions

All members contributed to the project. A final submission may specify contributions using the following categories:

- data acquisition and preprocessing,
- embedding and FAISS indexing,
- BM25 and hybrid retrieval,
- legal graph construction and graph expansion,
- evaluation and hyperparameter tuning,
- report writing and presentation.

Chapter 17. Conclusion

This project presents a reproducible Vietnamese Legal RAG retrieval system built around Qwen3 embeddings, BM25 lexical matching, weighted RRF fusion, and document-level legal graph expansion. The revised class-based implementation provides a unified entry point for index construction, hyperparameter tuning, multi-method evaluation, interactive search, and optional answer generation.

A major correction in the new graph architecture is the explicit hierarchy from documents to article nodes, plus document-level reference edges extracted from `link.to`. This improves graph expansion for article-level legal evidence while leaving appendix/table graph nodes as future work. The evaluation framework now compares five retrieval methods over 4,000 queries at five cutoffs and produces both aggregate and query-level outputs.

The generated CSV and JSON files provide the final numeric results for the report. Regardless of the final ranking, the framework enables clear diagnosis of whether errors originate from lexical retrieval, dense retrieval, fusion, graph mapping, graph expansion, or top-rank ordering. This modularity provides a sound foundation for future work on legal-domain fine-tuning, cross-encoder reranking, relation-aware graph reasoning, and expert-validated evaluation.

References

- [1] M. Doan, “Vietnamese Legal Documents,” Hugging Face dataset: `minhdoan17/vietnamese-legal-documents`.
- [2] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems*, 2020.
- [3] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, 2009.
- [4] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” *Proceedings of SIGIR*, 2009.
- [5] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Transactions on Big Data*, 2019.
- [6] T. Wolf, L. Debut, V. Sanh, et al., “Transformers: State-of-the-Art Natural Language Processing,” *Proceedings of EMNLP: System Demonstrations*, 2020.
- [7] Qwen Team, “Qwen Technical Report,” Alibaba Cloud, 2023–2025.
- [8] D. Edge, H. Trinh, N. Cheng, et al., “From Local to Global: A Graph RAG Approach to Query-Focused Summarization,” arXiv preprint, 2024.
- [9] V. Karpukhin, B. Oğuz, S. Min, et al., “Dense Passage Retrieval for Open-Domain Question Answering,” *Proceedings of EMNLP*, 2020.
- [10] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *Proceedings of EMNLP-IJCNLP*, 2019.
- [11] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, “LEGAL-BERT: The Muppets Straight Out of Law School,” *Findings of EMNLP*, 2020.